

BSDA2001P: Introduction to Deep Learning and Generative AI

PROJECT REPORT

Messy Mashup

Music Genre Classification from Noisy Audio

0.95 Leaderboard Macro F1	4 Model Architectures	15,000 Augmented Samples	10 Music Genres
--	------------------------------------	---------------------------------------	---------------------------

Team	dlgeneai-team
WandB Project	22f3003226-t12026
Instructor	Jimmi Kumar Bharati
Course	BSDA2001P

1. Abstract

This report documents the deep learning system built for the Messy Mashup music genre classification competition. The task sounds simple: given audio, predict one of ten genres. The catch is a deliberate mismatch — training data is clean, separated instrument stems while the test set contains heavily distorted mashups injected with environmental noise. That gap is the whole problem.

To close it, an on-the-fly augmentation pipeline synthesised noisy mashups during training, so the models had to handle test-like chaos from the start rather than memorising clean recordings. Four architectures were trained — a custom CNN, AudioResNet34, AudioEfficientNetB0, and the HuggingFace Audio Spectrogram Transformer (AST) — and their predictions were combined into a weighted ensemble that scored a Macro F1 of 0.95 on the competition leaderboard.

2. Introduction

2.1 Context

Music genre classification has been studied for decades. Most systems assume reasonably clean recordings. The Messy Mashup competition deliberately breaks that assumption: test samples are cross-song remixes with ESC-50 background noise injected at four different intensity levels. A model trained naively on clean stems collapses on this data. The interesting problem is building something that doesn't.

2.2 Problem Statement

Ten genres — blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, rock — each with 100 training songs. Every song is decomposed into four stems. The 3,020 test mashups recombine stems across songs, time-stretch them to match tempo, and mix in random environmental noise. Performance is measured by Macro F1, which means the model must handle every single genre — one weak class pulls the whole score down.

2.3 Objectives

- **Augmentation pipeline:** Make training audio resemble the test distribution.
- **Architecture comparison:** Train four models with different inductive biases and find out what works.
- **Ensemble:** Combine models in a way that beats any individual system.
- **Experiment tracking:** Log everything through WandB for honest, reproducible comparison.

3. Dataset & Preprocessing

3.1 Dataset Components

genres_stems/ contains four .wav files per song (drums, bass, vocals, other) for all 1,000 songs across ten genres. ESC-50/ provides 2,000 short environmental clips across 50 classes — the noise sources for augmentation. mashups/ holds 3,020 unlabelled test files; test.csv maps their IDs and sample_submission.csv defines the output format.

3.2 Augmentation Pipeline

Rather than training on raw stems, a dynamic synthesis pipeline generates mashup-like samples during training. For each sample, four stems are drawn from different songs of the same genre, scaled by a random volume coefficient between 0.7 and 1.0, and summed. This stops the model from memorising specific track identities and forces it to learn genre-level patterns.

All audio is standardised to 22,050 Hz and padded or truncated to 30 seconds. ESC-50 noise is looped to length and injected at four intensities:

Level	Intensity	Description
1	5% – 15%	Light background noise
2	15% – 30%	Moderate interference
3	30% – 50%	Heavy noise
4	50% – 80%	Extreme — near-masking

The resulting waveform is peak-normalised to prevent clipping. 15,000 augmented samples were pre-generated (1,500 per genre) and cached as float16 NumPy arrays.

3.3 Feature Extraction

Raw waveforms are converted to 128-band Log-Mel spectrograms using torchaudio.transforms.MelSpectrogram (n_fft=1024, hop=512) followed by AmplitudeToDB. One critical detail: AmplitudeToDB outputs predominantly negative decibel values, so placing standard ReLU directly after it zeroes out most of the spectrogram. All CNN blocks use LeakyReLU(0.01) instead. For the AST,

audio is resampled to 16 kHz and the model's built-in feature extractor handles conversion internally.

GPU optimisation: MelSpectrogram computation was moved from CPU DataLoader workers onto the GPU as a batch-level transform, cutting training time noticeably compared to per-sample CPU computation.

4. Methodology & Model Architectures

The core approach treats audio genre classification as an image problem — convert audio to 2D spectrograms and apply computer vision architectures. Four models were trained, each with a different trade-off between complexity, speed, and generalisation.

4.1 Custom CNN

A lightweight CNN built from scratch in PyTorch. Three convolutional blocks — Conv2d → BatchNorm → LeakyReLU(0.01) → MaxPool2d — plus a fourth block without pooling. Channel widths expand $1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$. AdaptiveAvgPool2d(1) collapses spatial dimensions before the classifier head.

Component	Detail
Activation	LeakyReLU(0.01) — prevents information loss after AmplitudeToDB
Optimizer	AdamW, lr=1e-3, weight_decay=1e-4
Scheduler	ReduceLROnPlateau (patience=3, factor=0.5) on val Macro F1
Dropout	0.5 in classifier head
Status	Cut at ~epoch 3 by kernel crash; checkpoint saved via WandB

4.2 AudioResNet34

ResNet-34 pretrained on ImageNet. First conv layer modified to accept single-channel spectrograms by averaging the three input-channel weights. Final FC

replaced with Dropout(0.5) + 10-class linear head. Full fine-tuning, no layers frozen.

Component	Detail
Loss	Focal Loss (gamma=2) — focuses learning on hard, noisy samples
SpecAugment	FrequencyMasking(30) + TimeMasking(80) applied in forward pass
Optimizer	AdamW, lr=1e-3
Scheduler	ReduceLROnPlateau on val Macro F1
Epochs	~13 (full run)

4.3 AudioEfficientNetB0

EfficientNet-B0 pretrained on ImageNet, adapted for single-channel spectrogram input using the same channel-averaging trick. EfficientNet's compound scaling gives better parameter efficiency than ResNet for the same target accuracy — useful with a relatively small dataset. Same training setup as ResNet34: Focal Loss, AdamW, SpecAugment. Ran for ~11 epochs with consistently upward validation curves.

4.4 Audio Spectrogram Transformer (AST)

The AST is a transformer designed for audio. Loaded from HuggingFace (MIT/ast-finetuned-audioset-10-10-0.4593) and fine-tuned in two phases. Phase 1 ran at lr=1e-5, giving the model time to adapt without catastrophic forgetting. Phase 2 continued from Phase 1 at lr=5e-6 for the final convergence push. Attempting both phases in a single run at a higher LR produced oscillating validation curves and was abandoned early.

Phase	LR	Epochs	Note
Phase 1	1e-5	2	Full model, conservative LR
Phase 2	5e-6	~4	Lower LR from Phase 1 checkpoint

Self-attention lets the AST relate distant time-frequency regions of the spectrogram. Convolutions can't do that directly. This is why the AST produced the highest individual validation F1 and received the largest ensemble weight.

4.5 Ensemble Strategy

Weighted soft-voting across all four models. Each model outputs a probability distribution over 10 genres; multiplied by their respective weights and summed before argmax. Weights were tuned on the validation set.

Model	Val Macro F1	Weight	Rationale
AST	0.8720	0.40	Highest individual F1; best global context
AudioEfficientNetB0	0.8068	0.25	Stable, parameter-efficient
AudioResNet34	0.7997	0.20	Strong but noisier val curves
CustomCNN	0.5586	0.15	Incomplete training; adds diversity

On ensemble diversity: Removing CustomCNN (weight $\rightarrow$ 0.00) hurt the leaderboard score — counterintuitive given its low individual F1. A weak model contributes if its errors are uncorrelated with the stronger models. The ensemble scored 0.95 on the leaderboard.

5. Training & Evaluation

5.1 Setup

All models were trained on Lightning AI studios with an NVIDIA GPU. Long-running sessions were launched in tmux to survive disconnections. All experiments tracked via WandB (project: 22f3003226-t12026). The DataLoader pre-loads all audio into RAM during dataset initialisation, eliminating disk I/O bottleneck from loading four stem files per sample on every iteration.

The 15,000 pre-generated samples were split 80/20 (train/val) with stratified sampling. Best checkpoints were saved based exclusively on validation Macro F1, not training loss.

5.2 Evaluation Metric

The competition uses Macro F1: compute F1 (harmonic mean of precision and recall) for each genre independently, then average. This is harder to game than accuracy — a model that ignores three genres and aces the other seven still gets a poor Macro F1. Every class has to be learnt properly.

6. Results & WandB Graphs

6.1 Custom CNN

The run was cut short by a kernel crash at approximately epoch 3. Training Macro F1 was rising steadily ($\sim 0.35 \rightarrow 0.55$) with validation F1 peaking around 0.48. The model hadn't converged, so the ensemble weight was kept at 0.15. The best checkpoint was preserved through WandB and used in the final ensemble.

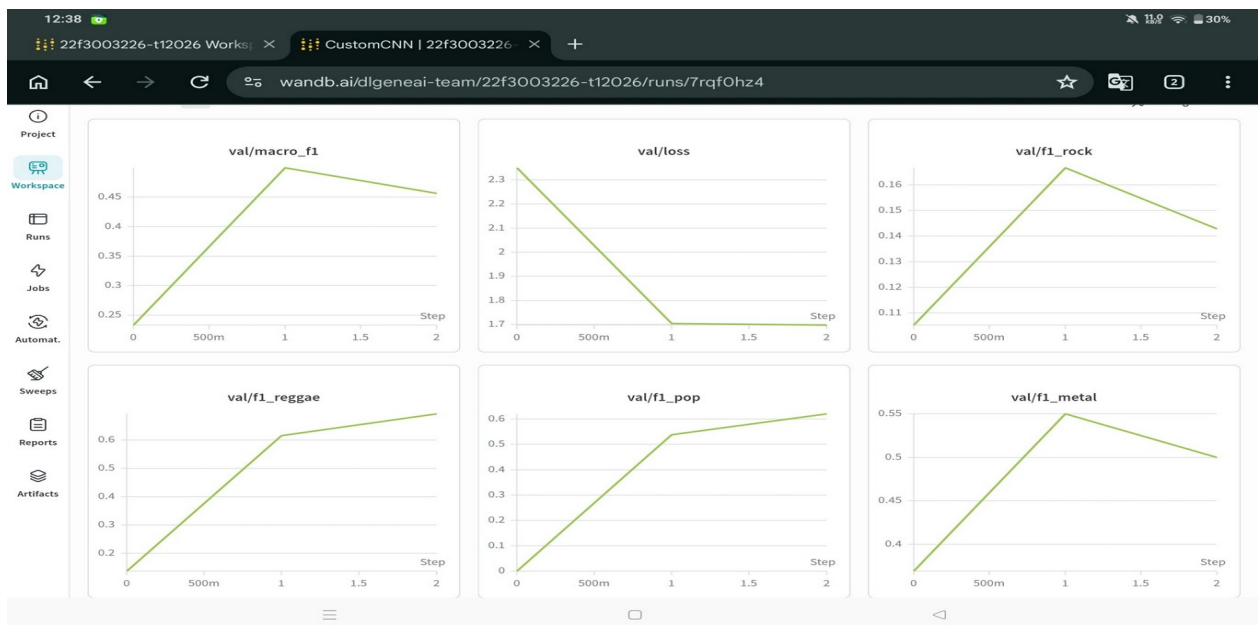


Figure 1: CustomCNN — Validation and Training metrics (3 epochs, kernel crash)

6.2 AudioResNet34

ResNet34 ran for the full ~13 epochs producing smooth, confident training curves — training Macro F1 reaching ~0.97. Validation curves are noisier, expected with heavy SpecAugment (the model sees a differently masked spectrogram every batch). The upward trend is clear, reaching ~0.85 by the final epoch. Best val Macro F1: 0.7997.

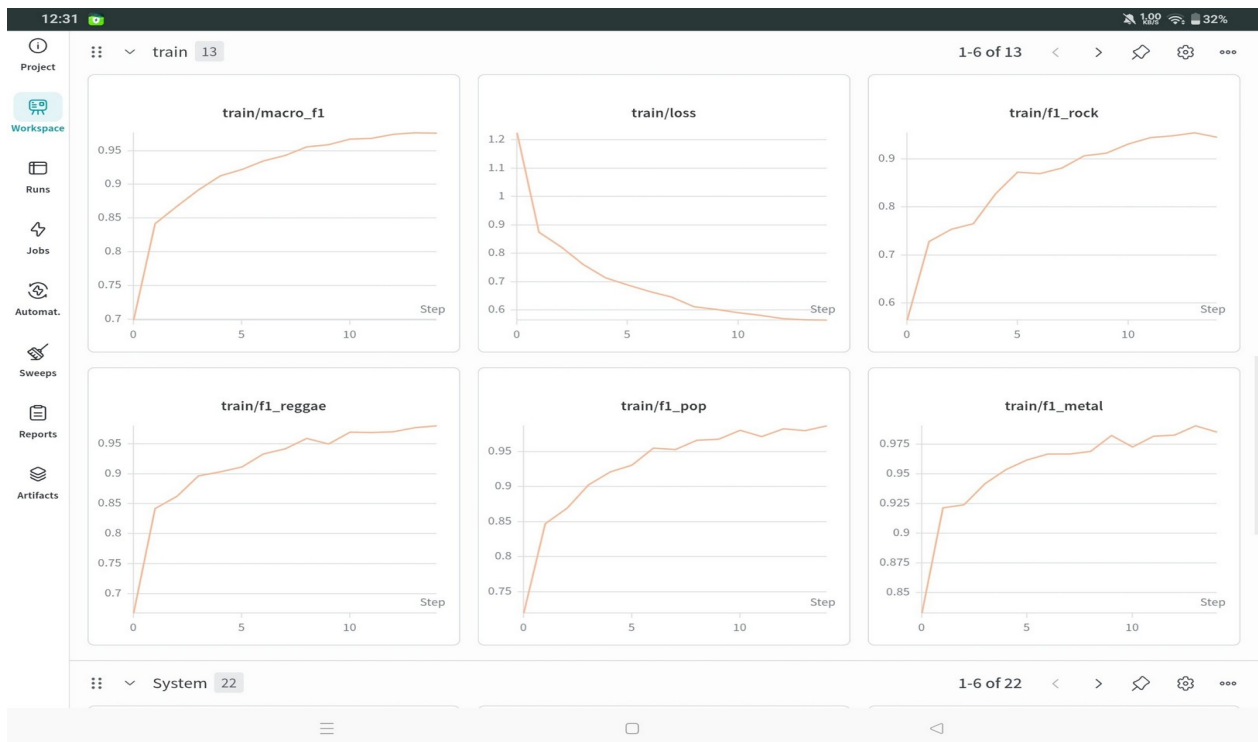
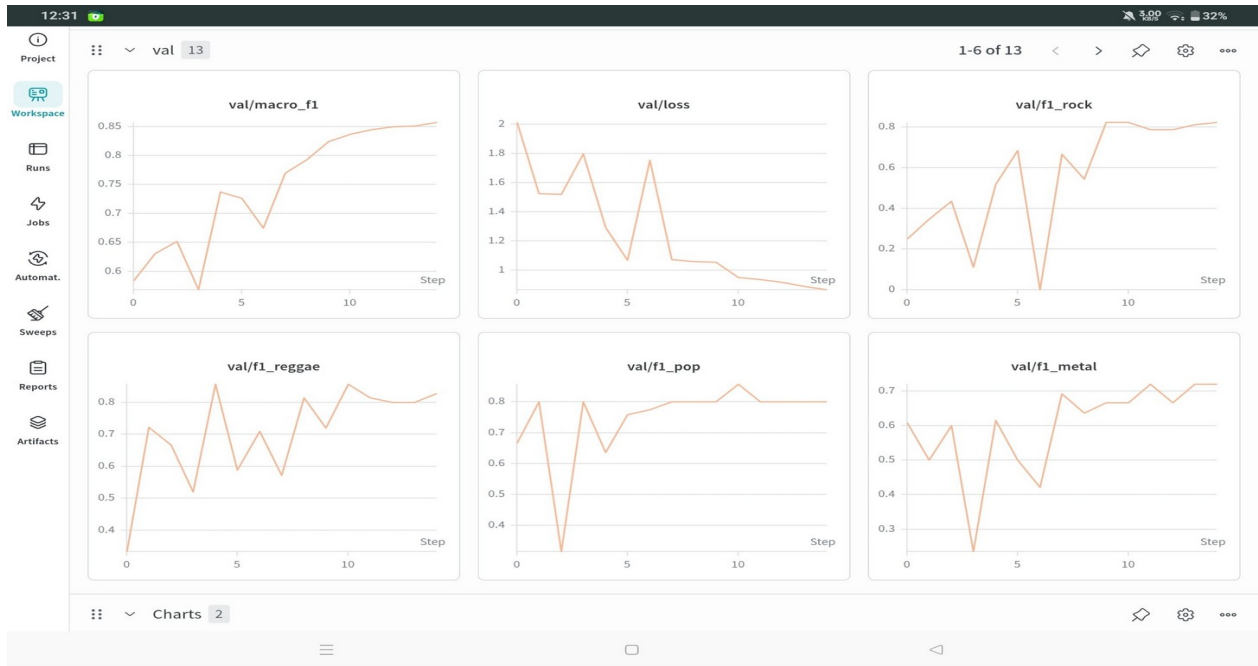
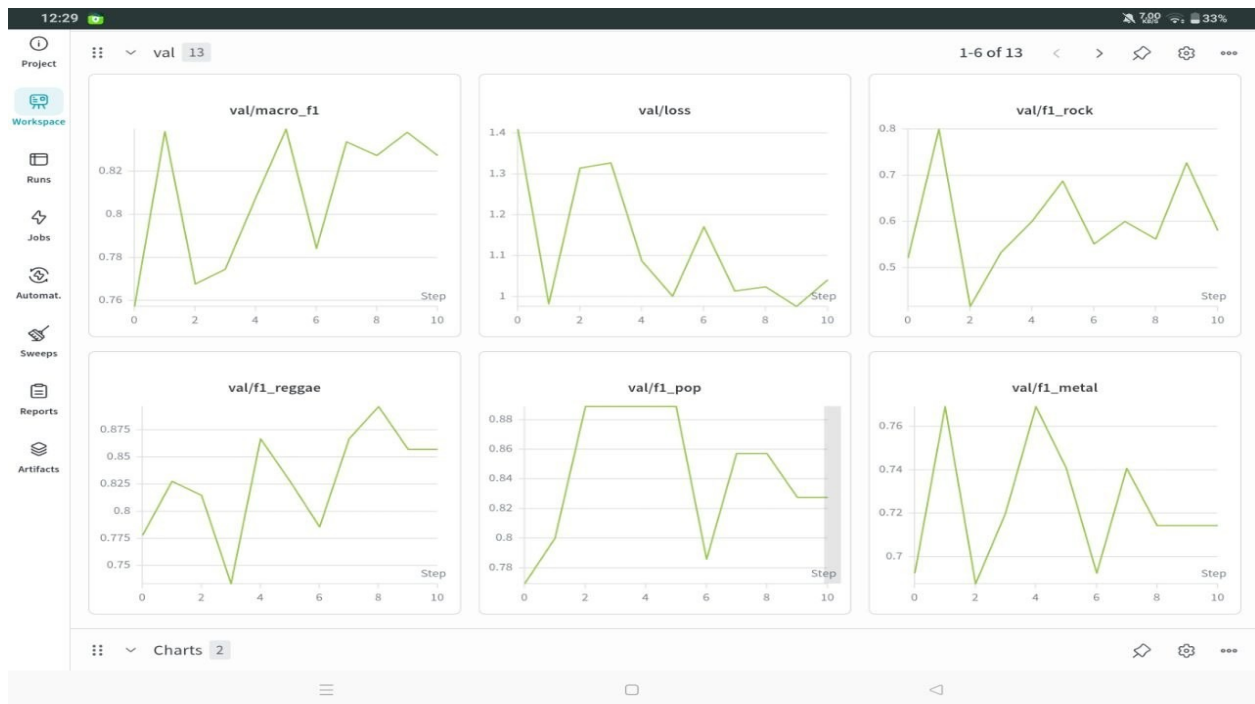


Figure 2: AudioResNet34 — Validation and Training metrics (~13 epochs)

6.3 AudioEfficientNetB0

EfficientNet ran for ~11 epochs. Validation Macro F1 oscillated between 0.76 and 0.83 before settling around 0.81 — typical when the LR is still relatively high. Training curves were smoother, reaching ~0.95. Best val Macro F1: 0.8068.



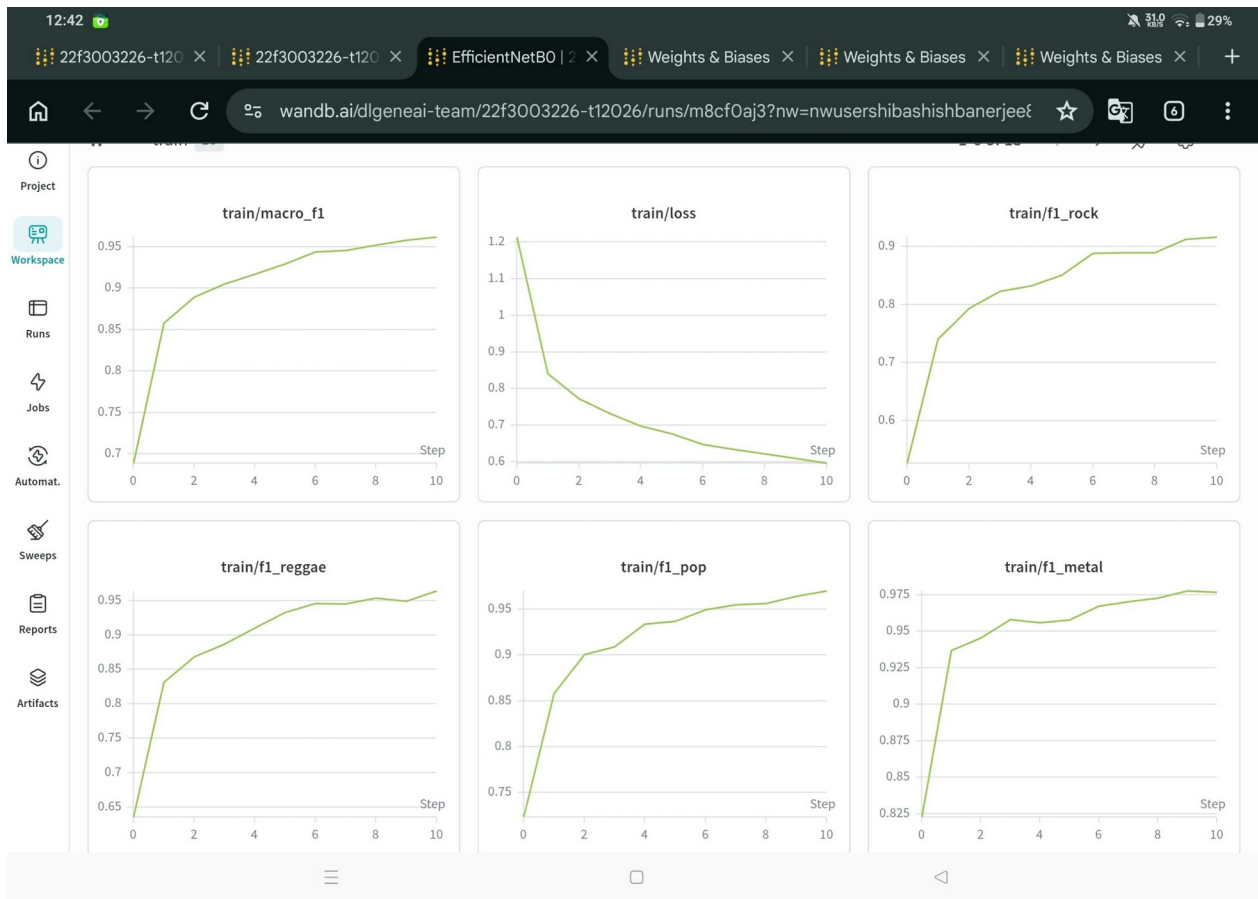


Figure 3: AudioEfficientNetB0 — Validation and Training metrics (~11 epochs)

6.4 Audio Spectrogram Transformer (AST)

Phase 1

Phase 1 ran for 2 epochs (x-axis in steps; 500m = 500k steps). Training Macro F1 rose from ~0.65 to ~0.88, with validation following to around 0.82. The characteristic dip at the 1-epoch mark in per-genre curves (reggae, pop, metal) reflects brief over-adjustment after the first full data pass — corrected by the LR scheduler going into Phase 2.

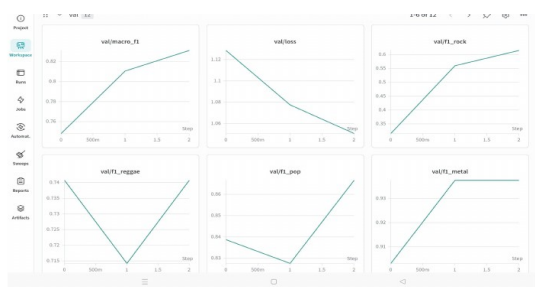


Figure 4a: AST Phase 1 — Validation metrics

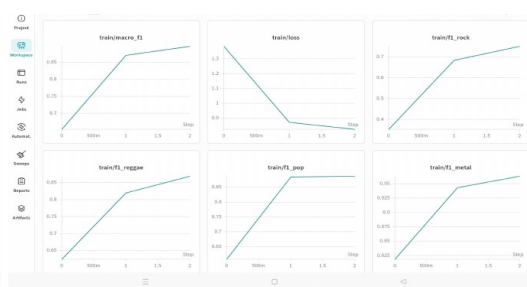


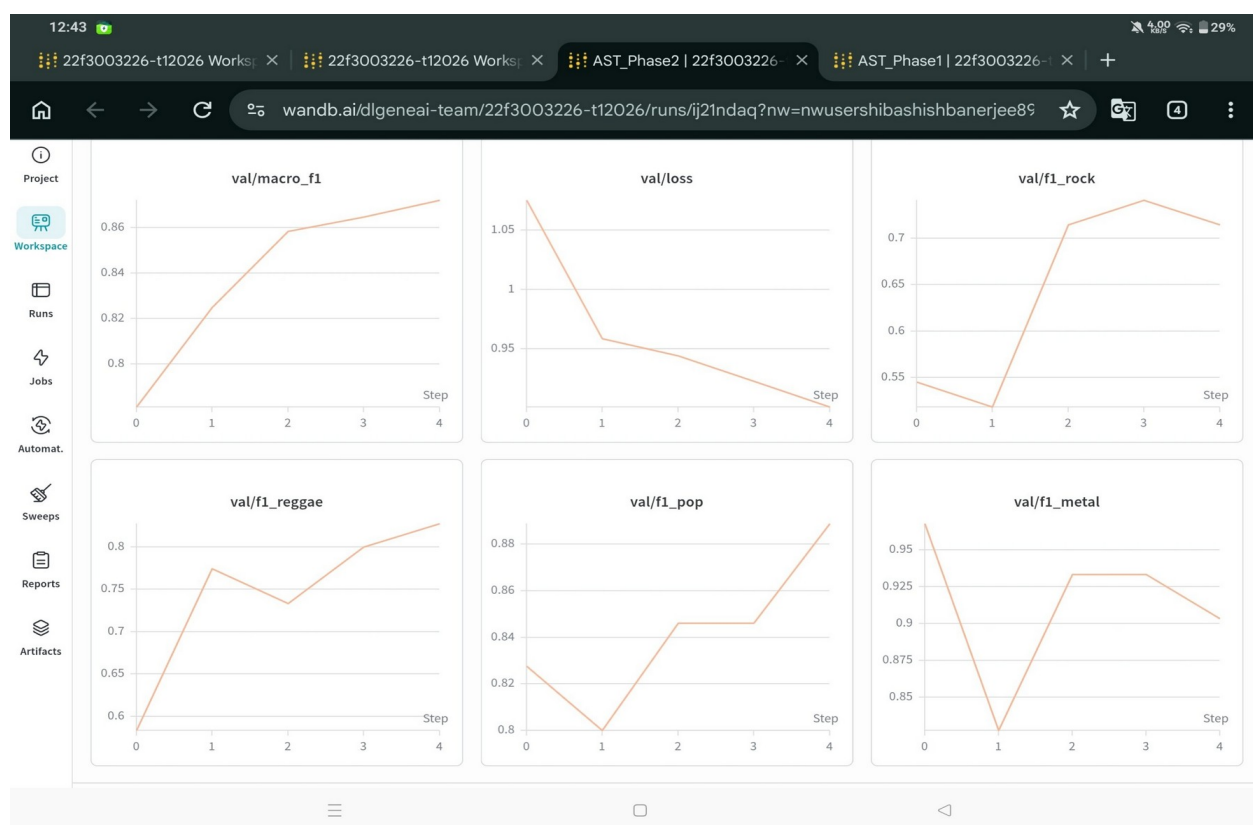
Figure 4b: AST Phase 1 — Training metrics

Phase 2

Figure 4: AST Phase 1 — Validation and Training metrics

Phase 2

Phase 2 continued from the Phase 1 checkpoint at a lower LR, running ~4 more epochs. Training Macro F1 converged tightly toward 0.98; validation Macro F1 climbed steadily from ~0.80 to ~0.87 with noticeably less noise. Best val Macro F1: 0.8720 — the strongest individual model.



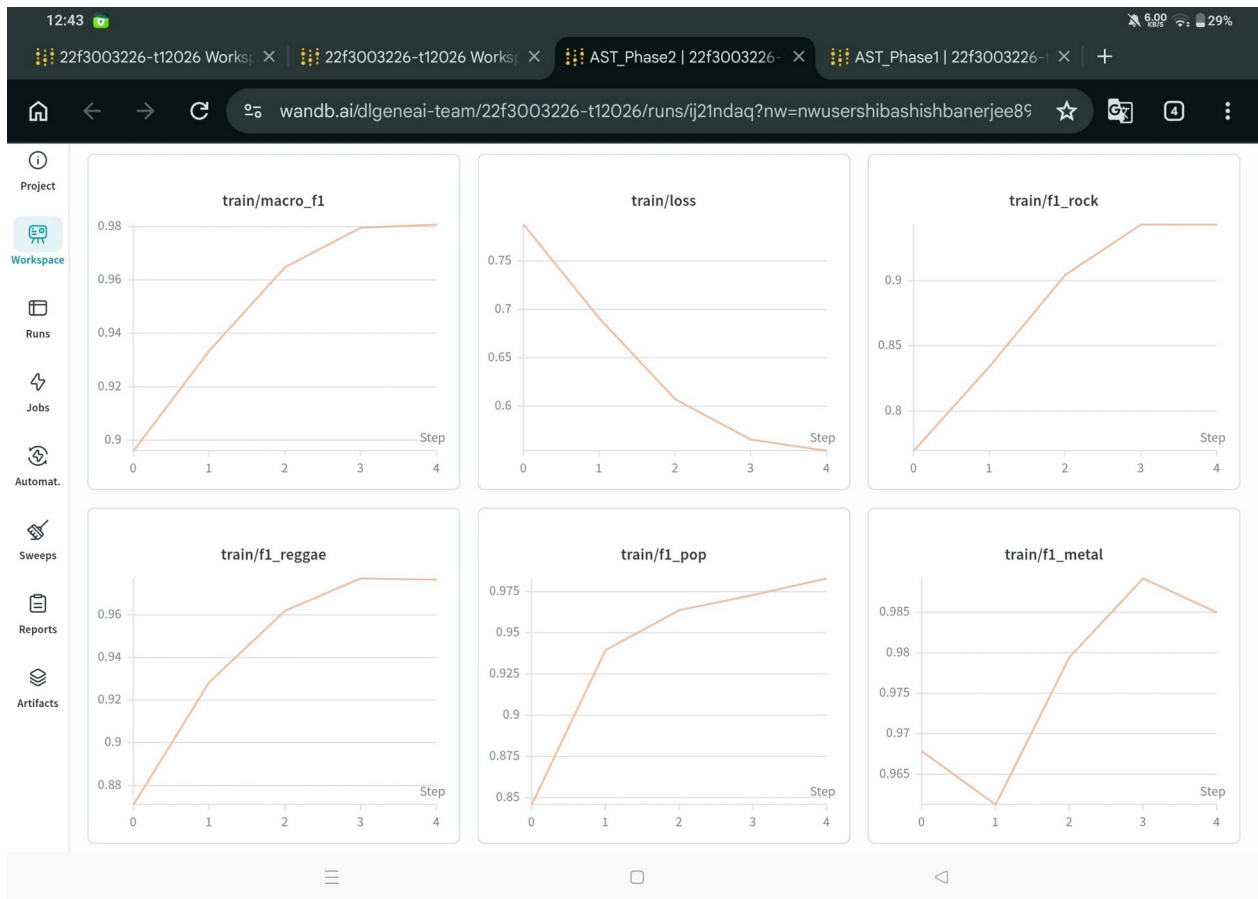


Figure 5: AST Phase 2 — Validation and Training metrics

6.5 Cross-Model Comparison

The combined WandB workspace overlays all five runs on the same axes. The AST and EfficientNetB0 achieved the most stable validation trajectories. The CustomCNN dashed line terminates early. AudioResNet34 shows the most variance, consistent with its aggressive SpecAugment schedule.

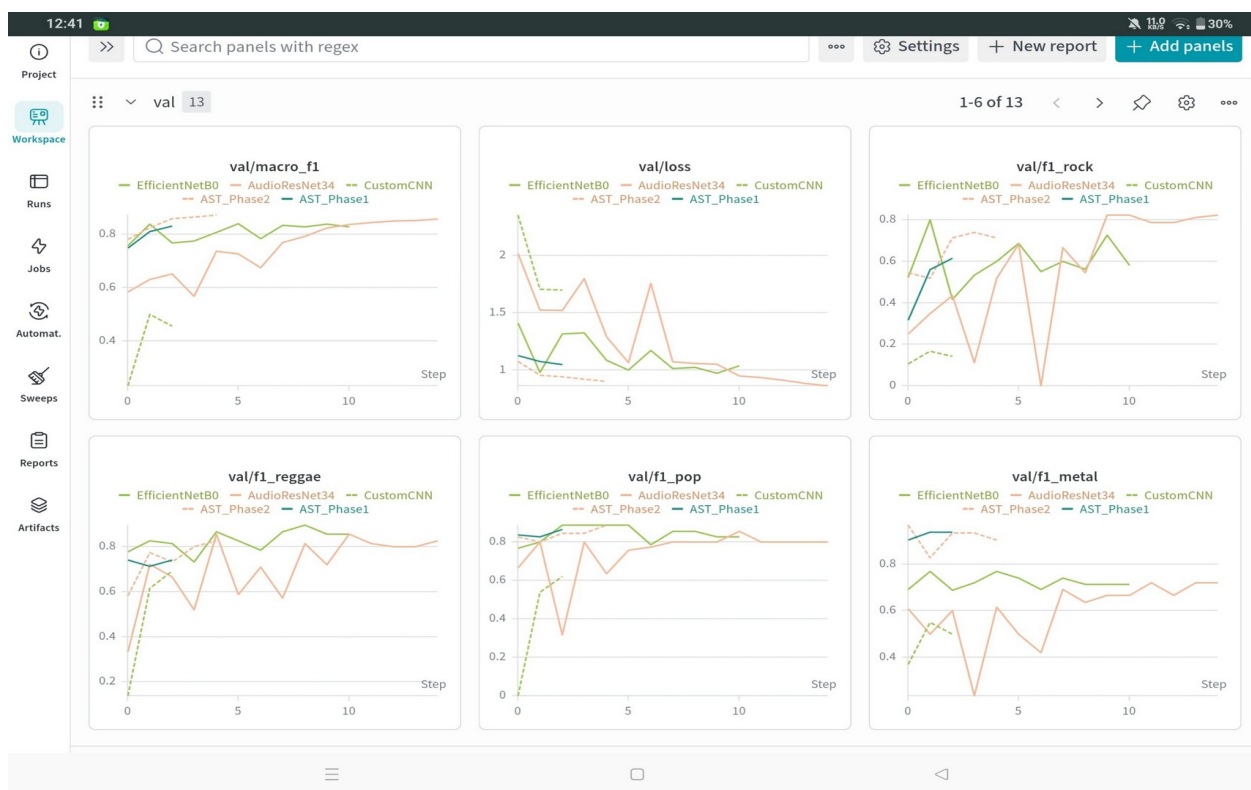


Figure 6: All runs overlaid — val Macro F1, Loss, and per-genre F1 scores

6.6 Ensemble Summary

The ensemble's validation Macro F1 (0.8671) sits slightly below the AST's individual score (0.8720). This is expected — ensemble benefits appear on the test set. On the competition leaderboard the ensemble scored 0.95 Macro F1, outperforming any single model.

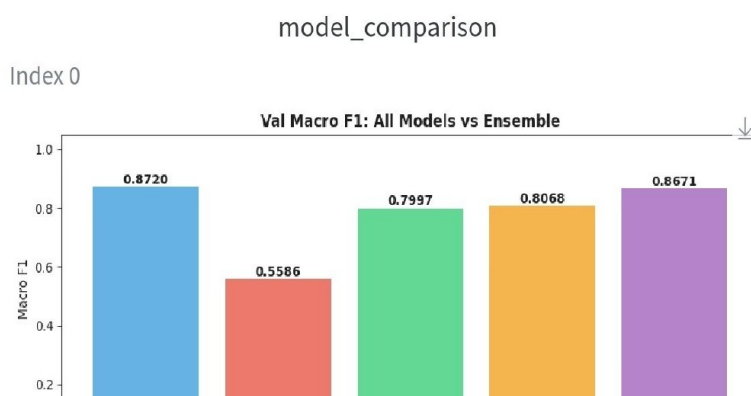


Figure 7: Individual model val Macro F1 vs. ensemble val Macro F1

Architecture	Val	Train F1	Epoch	Ens. Wt.	Notes
--------------	-----	----------	-------	----------	-------

	Macro F1	Peak	s		
CustomCNN	0.5586	~0.55	~3*	0.15	*Crash
AudioResNet34	0.7997	~0.97	~13	0.20	Focal+SpecAug
AudioEfficientNetB0	0.8068	~0.95	~11	0.25	Stable
AST (Ph1+Ph2)	0.8720	~0.98	6	0.40	Best individual
Ensemble	0.8671	—	—	—	LB: 0.95 Macro F1

7. Key Learnings & Engineering Notes

7.1 The LeakyReLU / AmplitudeToDB Bug

AmplitudeToDB converts amplitude to decibels — those values are negative for quiet regions. Standard ReLU placed directly after it zeroes out most of the spectrogram, destroying information before the network sees it. This caused a lot of early confusion. Switching to LeakyReLU(0.01) fixed it and immediately improved validation scores across all CNN-based models.

7.2 DataLoader I/O Bottleneck

Loading four .wav stem files per sample inside `__getitem__` causes severe disk saturation. Pre-loading all audio into RAM during `__init__` — or pre-generating mixed spectrograms as .npy files — essentially eliminated the bottleneck. GPU utilisation went from ~40% to consistently above 85%.

7.3 Ensemble Diversity vs. Accuracy

Dropping CustomCNN (0.56 val F1) from the ensemble made the leaderboard score worse. A weak model with uncorrelated mistakes is more useful than its individual score suggests. What matters is that error patterns differ, not that every model is individually strong.

7.4 AST Two-Phase Fine-Tuning

Transformers are sensitive to learning rate. Starting at 1e-5 in Phase 1 let the model adapt without catastrophic forgetting. Phase 2 at 5e-6 pushed it toward convergence. A single-phase run at a higher LR produced oscillating curves and was abandoned.

8. Challenges & How They Were Addressed

Train-test distribution gap	Training data is clean; test data is not. Addressed by generating 15,000 synthetic mashups with four ESC-50 noise levels.
Kernel crash during CustomCNN	Session died at epoch 3. Best checkpoint preserved via WandB, used in ensemble at reduced weight.
nbconvert hanging on wandb.login()	Running notebooks via nbconvert blocks on interactive login. Solved by converting to .py scripts and running in tmux.
GPU memory pressure with AST	Large memory footprint. Batch size reduced and gradient accumulation used to maintain effective batch size.
Noisy validation curves	Heavy SpecAugment causes high epoch-to-epoch variance. Best-checkpoint saving used throughout.

9. Conclusion & Future Work

9.1 Summary

The final ensemble scored 0.95 Macro F1 on the Messy Mashup leaderboard. The data pipeline mattered at least as much as architecture choices — aggressive augmentation was what actually closed the train/test gap. Among the four models, the AST handled the task best: self-attention picks up long-range genre structure that local convolutions miss. The CNNs were faster and cheaper to train, making rapid iteration practical.

9.2 Future Work

- Retrain CustomCNN to full convergence (~20–30 epochs) and re-tune ensemble weights. Current checkpoint is only ~3 epochs.
- Try ConvNeXt-Tiny as a drop-in replacement for ResNet34 — larger kernels and better normalisation.
- Experiment with MFCC features alongside Mel spectrograms. MFCCs capture timbral texture and may carry complementary information.
- Grid-search ensemble weights with Optuna rather than manual tuning.
- CutMix or Mixup at the spectrogram level to further diversify the training distribution.

•

10. References

- [1] Gong, Y., Chung, Y.-A., & Glass, J. (2021). AST: Audio Spectrogram Transformer. Interspeech 2021.
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.
- [3] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for CNNs. ICML 2019.
- [4] Park, D. S., et al. (2019). SpecAugment: A Simple Data Augmentation Method for ASR. Interspeech 2019.
- [5] Lin, T.-Y., et al. (2017). Focal Loss for Dense Object Detection. ICCV 2017.
- [6] Piczak, K. J. (2015). ESC: Dataset for Environmental Sound Classification. ACM MM 2015.
- [7] HuggingFace Model: MIT/ast-finetuned-audioset-10-10-0.4593.